

担当箇所の基本設計・詳細設計書_ver 光同期廃止後

25G1113 廣岡花 グループ7

2026 年 6 月 22 日

1 担当箇所の基本設計

1.1 機能一覧

図 1 に担当箇所である同期制御機能を中心に詳細化した FBS 図を示す。また表 1 に、各機能の概要と実装上の要点をまとめる。本システムは、1 台の PC (Processing) を指揮者 UI、1 台の Master Arduino を制御ハブ、4 台の Slave Arduino を各楽器ノードとして構成し、Processing から受け取った演奏設定を Master が集約して各 Slave へ配信することで、複数ノードの演奏を協調制御する。指揮者 UI (UI.pde) は、ユーザーからの入力として、BPM の増減 ([↑] / [↓] キー)、演奏順の登録 ([1]~[3] キー)、演奏順のリセット (ボタン操作)、共通オクターブの増減 (ボタン操作)、および設定確定・送信 (ボタン操作) の 5 種類を受け付ける。これらの入力内容は画面上の各エリアにリアルタイムに反映され、設定確定・送信ボタンが押されると、その時点のオクターブ・演奏順・BPM をまとめた 1 回限りの初期設定情報が Master へ送信される。送信後は演奏中とみなし、以後は BPM の変更のみを受け付けて他の設定変更を受け付けられない状態 (UI ロック) に切り替わる。これは、演奏中に演奏順やオクターブが変更されてしまうことを防ぐためである。演奏制御のロックおよび解除は、Master からの通知に基づいて行う。具体的には、Master から演奏開始の通知 (START) が送信されると、指揮者 UI は BPM 以外の操作を即座にロックし、演奏が終了した通知 (FINISH) を受信すると、ロックを解除して通常の編集可能な状態に戻す。なお、現段階では Master 実機との接続が無い環境での動作確認用に、キーボードの [s] キーを押すことでも同様にロックを解除できる簡易的な仕組みを設けている。さらに、本バージョンでは指揮者 UI と同一プロセス上に、BPM と演奏状態を可視化するための別ウィンドウ (BearWindow) を追加した。BearWindow は指揮者 UI 側から毎フレーム BPM と演奏中かどうかの状態を受け取り、ドット絵画像によるクマの横スクロールアニメーションとしてこれを表示する。BPM が速いほど画面のスクロール速度が増すため、演奏のテンポを視覚的に把握できる。操作はキーボードの [Space] キーのみであり、演奏中に [Space] キーを押すとクマがジャンプし、飛んでいるコインに触れると得点 (獲得コイン数) が加算される。逆に地面の障害物に接触すると、クマがダメージを受けた見た目

に変化し、獲得コイン数が一定数減少する。演奏が開始されると同時に今回分の得点集計が始まり、Master から演奏終了の通知（FINISH）を受信するとゲームを終了し、その回の獲得コイン数とこれまでの上位記録をランキングとして指揮者 UI 側に表示する。ただし、このランキングは Processing の実行中のみ保持される一時的な記録であり、Processing を停止すると消去される。

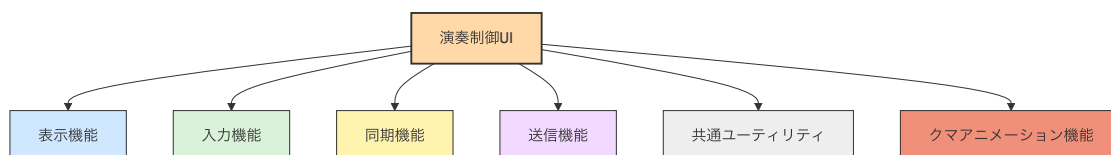


図 1 UI 面の同期・表示に着目した FBS

表 1 演奏制御 UI の機能一覧

区分	内容
表示機能	タイトルおよび送信ボタンの描画
	BPM 設定エリアの描画
	演奏順番設定エリアの描画
	共通オクターブ設定エリアの描画
	Slave 状態ボックスの描画
	送信内容および現在の状態表示
入力機能	↑/↓キーによる BPM 変更, 1～3 キーによる演奏順登録
	設定送信, 順番リセット, オクターブ変更ボタンの操作
演奏制御 ロック機能	Master から演奏開始通知 (START) を受信した時点で BPM 以外の操作を即座にロック
	Master から演奏終了通知 (FINISH) を受信した時点でロックを解除し, 編集可能状態へ復帰
	実機未接続時の動作確認用として, [s] キー押下によるロック解除を仮実装
送信機能	オクターブ・演奏順・BPM を含む初期設定情報の生成
	初回の設定送信および演奏中の BPM 変更送信
共通 ユーティリティ	ボタン領域のホバー判定
	演奏順および選択状態の初期化
クマアニメー ション機能	ドット絵画像 (歩行 2 種・ジャンプ・ダメージ) によるクマの表示と状態切替
	BPM に連動したスクロール速度の制御 (空・地面・雲・木・コイン・障害物)
	[Space] キーによるジャンプ操作とコイン収集
	障害物接触時のダメージ表示と獲得コイン数の減少
	Master から演奏終了通知 (FINISH) 受信時のゲーム終了とランキング表示 (Processing 停止で記録は消去)
	ランキングおよびベストスコアの保持・更新

1.2 具体的な実装内容

1.2.1 UI 設定入力



図2 Processing のUI画面のプロトタイプ

図2に本システムの実際のUI画面のプロトタイプ、表3にUI画面上の主要な操作要素と、それぞれの操作方法および演奏中の挙動を示す。本システムのUI画面は、BPM設定エリア、演奏順番設定エリア、共通オクターブ設定エリアの3つの操作領域と、画面右上に配置した設定確定・送信ボタンから構成される。各操作領域は、現在演奏中であるかどうかに応じて編集可否が切り替わり、ロック中は背景色を淡いグレーに変化させるとともに、該当ボタンを非活性表示にすることで、操作可能な項目を視覚的に明示している。BPM設定エリアでは、キーボードの[↑]キーを押すたびにBPMを10加算し、[↓]キーを押すたびに10減算する。BPMは30から180の範囲に制限されており、この範囲を超える入力は無視される。BPMのみは演奏中であっても変更を許可しており、テンポを動的に調整できる設計としている。演奏順番設定エリアでは、楽器番号に対応するキー([1]:ピアノ, [2]:フルート, [3]:木琴)を押した順に演奏順を確定する。ドラムは演奏開始から常時演奏されるため、演奏順設定の対象から除外している。設定した順番は画面中央に「1番手」「2番手」「3番手」の形式で表示され、未選択の楽器は「未選択」と表示される。また、画面右側に配置した順番リセットボタン(クリック操作)を押すことで、演奏順番の選択状態を初期化で

きる。演奏中はこの操作領域全体がロックされ、キー入力を受け付けないとともに、リセットボタンの表示も「ロック中」に変化する。共通オクターブ設定エリアでは、画面下部に配置した [▲] ボタンおよび [▼] ボタンのクリック操作によってオクターブを1ずつ変更する。オクターブは国際式表記で-1から9の範囲とし、上限・下限を超える操作は無視される。演奏中はオクターブの加減ボタンが非活性表示となり、変更できない状態になる。

表 3 UI 画面における主要操作要素と挙動

UI 要素	操作方法	演奏中の挙動
設定確定・送信ボタン	クリックにより設定をパケット化して Master へ送信する	常に有効。初回は 8 バイトの初期パケット，2 回目以降は BPM3 バイトの差分パケットを送信する
BPM 設定	[↑] キーで +10, [↓] キーで -10 (範囲：30～180)	演奏中も変更可能
演奏順番設定	[1]～[3] キーを演奏したい順に押下し、ピアノ・フルート・木琴の演奏順を確定する	ロックされ入力を受け付けない
順番リセットボタン	クリックにより演奏順番の選択状態を初期化する	ボタン表示が「ロック中」に変化し操作不可となる
オクターブ [▲] ボタン	クリックでオクターブを +1 する (上限 9)	非活性表示となり操作不可となる
オクターブ [▼] ボタン	クリックでオクターブを -1 する (下限 -1)	非活性表示となり操作不可となる

1.2.2 Master へのシリアル送信処理

図 3 に、Processing から Master への送信処理における状態遷移を示す。Processing 側では、待機中の状態において BPM、演奏順、オクターブの各設定を受け付け、設定確定・送信操作が行われた時点でパケット生成処理へ遷移する。オクターブは -1~9 の範囲で入力し、送信時には 00~10 の 2 桁固定長文字列に変換する。演奏順は楽器 1~3 に対応するキー入力をもとに 3 桁の文字列として保持し、BPM は 30~180 の範囲で 3 桁固定長の値として扱う。これらの値はオクターブ 2 桁・演奏順 3 桁・BPM 3 桁の順に連結し、Master へ送信可能な固定長の文字列形式に整形する。送信処理では、初回送信時と 2 回目以降で通信内容を切り替える。初回送信では、オクターブ、演奏順、BPM を含む 8 バイトの初回設定パケットに改行を付加して送信する。一方、2 回目以降は BPM のみを 3 バイトの差分パケットとして送信し、演奏中の通信負荷を抑制する。この分岐は、初回の設定送信が完了したかどうかを内部で記憶しておくことで管理する。また、Processing 側では Master からの応答をシリアル受信イベントとして受け取り、状態に応じて UI ロック制御を行う。Master から演奏開始の通知 (START) を受信した場合は演奏中のロック状態へ移行し、演奏順およびオクターブの変更を禁止する。演奏終了の通知 (FINISH) を受信した場合はロックを解除し、再び待機中の編集可能状態へ戻る。この制御により、演奏中の誤操作を防ぎつつ、BPM のみを動的に変更できる入力インターフェースを実現している。

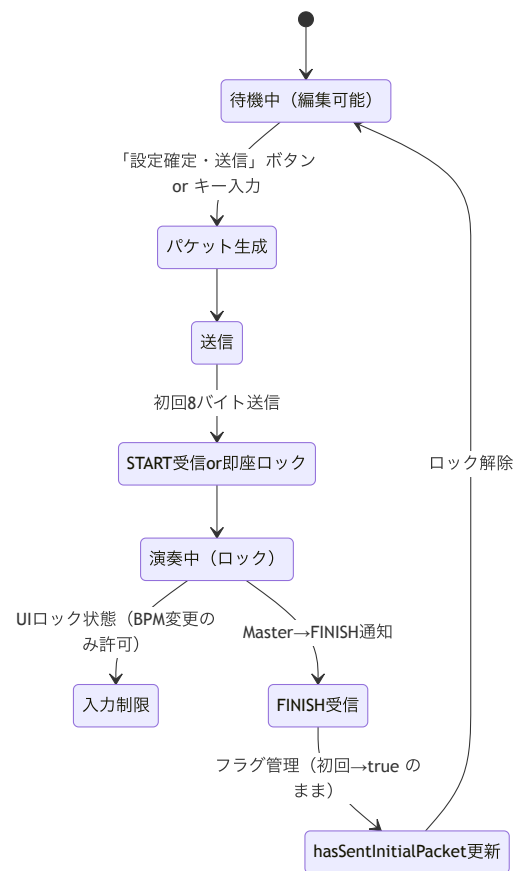


図 3 Processing から Master への送信の状態遷移図

1.2.3 クマアニメーション (BearWindow) の演出制御

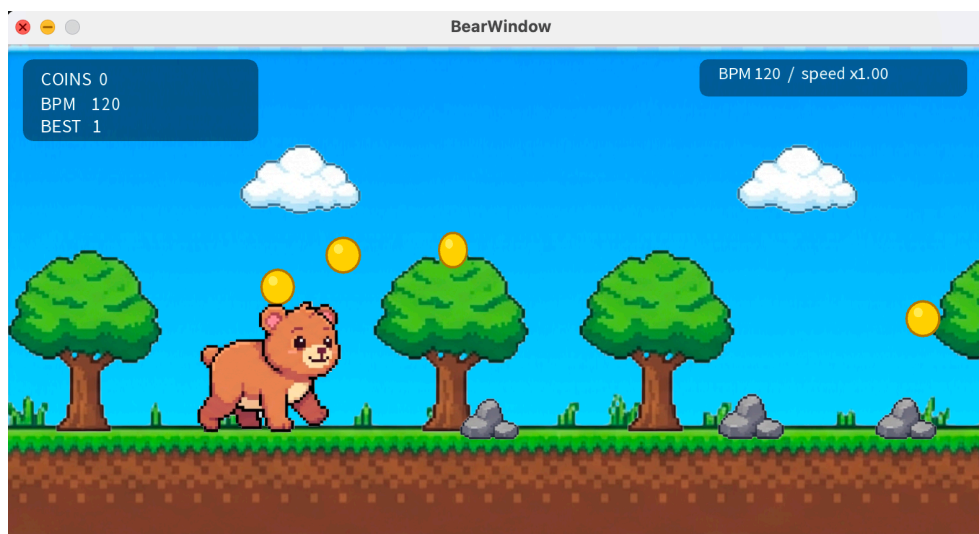


図4 クマアニメーション (BearWindow) の表示画面のプロトタイプ

図7に、演奏中に表示されるクマアニメーションウィンドウの画面のプロトタイプ、図8にクマアニメーションの状態遷移図を示す。本ウィンドウは指揮者 UI とは同一プロセス・別ウィンドウとして、指揮者 UI の起動時に同時に立ち上げる構成とした。これにより、1つの操作（演奏開始）に対して、指揮者 UI 画面とクマアニメーション画面という2つの表示が独立して並行に動作する。両者の連携は一方向のみとし、毎フレーム指揮者 UI 側から現在の BPM と演奏中かどうかの状態をクマアニメーション側へ渡し、クマアニメーション側はこれらの値だけを参照して描画とゲーム進行を行う。逆方向（クマアニメーション側から指揮者 UI 側）への影響は、ランの結果（獲得コインとランキング）の通知のみに限定し、システム全体の制御がクマアニメーションの都合で乱れないようにしている。描画する画像（クマの歩行2種・ジャンプ・ダメージ、および空・地面・雲2種・木2種・石）はすべてドット絵の PNG として用意した。これらの画像は指揮者 UI 側でまとめて読み込んだうえでクマアニメーション側に渡し、クマアニメーション側では画像の読み込み処理を行わない方針とした。これは、クマアニメーション側のウィンドウ内で直接画像を読み込む

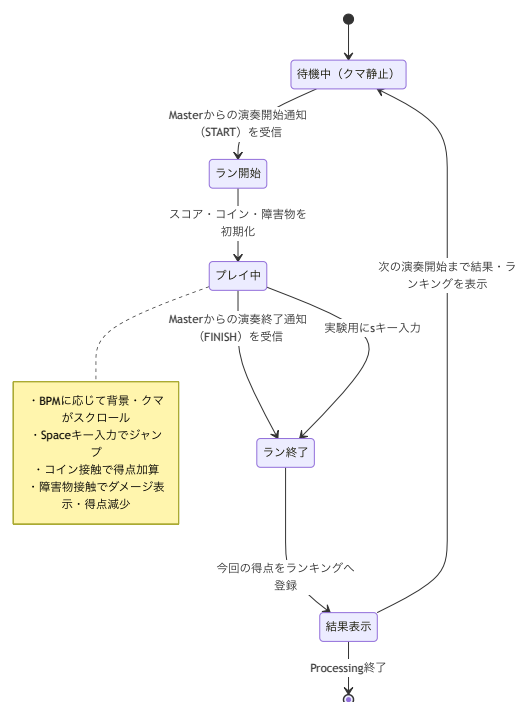


図5 クマアニメーションの状態遷移図

と、実行環境によっては画像ファイルの参照先を正しく認識できず、読み込みに失敗する可能性があるためであり、読み込み処理を1か所に集約することで、どの環境でも安定して起動できるようにするための工夫である。クマアニメーション内の処理は、毎フレームの状態更新と描画の2段階に分けて実装した。状態更新では、現在のBPMに応じてスクロール速度を決定し、雲・木・地面・コイン・障害物の位置を進める。スクロール速度はBPMに比例して大きくなるため、演奏のテンポが速いほどクマの歩行アニメーションおよび背景の流れが速くなり、テンポ感を視覚的に把握できる。地面の画像は横方向に連続して並べ、画面の左端まで流れた分を右端へ循環させることで、継ぎ目を視認させずに無限スクロールしているように見せている。操作はキーボードの[Space]キーのみであり、演奏中に[Space]キーを押すとクマがジャンプする。ジャンプは空中にいる間と接地している間で見た目を切り替え、重力を模した動きで自然に落下して着地する。クマの見た目は、歩行・ジャンプ・ダメージの3状態を毎フレーム判定して切り替え、歩行中はテンポに合わせて2種類の歩行画像を交互に表示する。障害物に接触した直後は一定時間ダメージの見た目に変化させ、連続してダメージを受けないよう短時間の無敵時間を設けている。当たり判定は、画面に表示する画像の見た目の大きさとは別に、クマ・コイン・障害物それぞれに専用の判定領域を設ける方式とした。クマの判定領域は画像のおおよその中心に置いた円形とし、コインとの当たり判定は距離による判定、障害物との当たり判定は円と四角形の重なり判定によって行う。この方式により、見た目の画像サイズを後から調整しても当たり判定の挙動に影響を与えずに済む。コインに触れると獲得コイン数が加算され、障害物に触れると獲得コイン数が一定数減算される。演奏が開始されると、クマアニメーション側はその変化を検知して今回分の得点集計（ラン）を自動的に開始する。演奏が終了すると、指揮者UI側からクマアニメーション側へ終了が通知され、当該ランの結果がランキングに登録される。指揮者UI側はクマアニメーション側に結果があるかどうかを確認し、結果がある場合は獲得コインとランキングを指揮者UI画面上にも表示する。

2 担当箇所の詳細設計

2.1 ソフトウェア設計

2.1.1 全体の処理フロー

システム全体のフローチャートを図 6, 図 7 に示す。本処理は、Processing の起動時に `setup()` が実行され、シリアルポートを 9600bps で初期化するところから開始する。接続に成功した場合は `isSerialConnected` を `true` に設定し、失敗した場合はシミュレーションモードとして `isSerialConnected=false` のまま UI 初期化を完了する。続いて `setup()` 内では、クマアニメーション用の別ウィンドウ `BearWindow` を生成し、クマおよび背景・地形・障害物に用いる 11 種類のドット絵画像をメインスケッチ側で `loadImage()` した上で `bearWin` へ受け渡し、最後に `PApplet.runSketch()` によって `BearWindow` を起動する。これにより、指揮者 UI とクマアニメーションは同一プロセス内の 2 枚の独立したウィンドウとして並行に動作する構成となる。初期化完了後は `draw()` によるメインループに入り、UI の描画とユーザー操作の受付を毎フレーム繰り返すとともに、現在の BPM と演奏中フラグ `isPlaying` を `bearWin.bearBpm`, `bearWin.bearPlaying` へ書き込み、`BearWindow` 側の描画内容と同期する。ユーザー操作は、BPM 変更、演奏順設定、オクターブ変更、送信、ジャンプの 5 系統に分岐する。BPM は `↑/↓` キー入力により 30~180 の範囲で更新される。演奏順は 1~3 キー入力により設定されるが、`isPlaying=true` の間には変更できず、待機中のみ `playOrder[]` へ反映される。オクターブもボタンクリックによって -1~9 の範囲で更新されるが、同様に演奏中は操作を無効化する。スペースキーが押下されると、押し続け判定用フラグ `bearJumpKeyHeld` によって 1 回の押下につき 1 度だけ `bearWin.triggerJump()` が呼び出され、`BearWindow` 側のクマがジャンプする。送信ボタンが押されると `sendParametersToMaster()` が呼び出され、Master への送信処理へ遷移する。送信処理では、`hasSentInitialPacket` の値によって初回送信と 2 回目以降の送信を切り替える。初回送信では、オクターブ 2 桁、演奏順 3 桁、BPM 3 桁を連結した初回パケットを `buildPacket()` で生成し、シリアル接続時は Master へ送信する。送信完了後は `hasSentInitialPacket` を `true` に更新し、`isPlaying=true` として UI をロックする。この `isPlaying` の変化は `BearWindow` 側へ同期され、`BearWindow` の `draw()` が変化を検知して `startRun()` を呼び出すことで、今回のランを開始し、コイン・障害物・スコアを初期化したうえで BPM に応じたスクロール演出を開始する。2 回目以降の送信は BPM のみを 3 桁で送信し、演奏中のテンポ変更に対応する。また、Master からの状態通知は `serialEvent()` で受信する。START または PLAYING を受信した場合は `isPlaying=true` を維持してロック状態を継続し、FINISH または STOP を受信した場合は `completeCurrentRun()` を呼び出して `isPlaying=false` とするとともに、`BearWindow` 側の `completeRun()` を実行してランを終了させ、獲得コインをランキングに登録する。シリアル未接続のシミュレーションモードでは、送信ボタン押下時に擬似的にロックし、s キー押下で同様に `completeCurrentRun()` を呼び出すことでランの終了を代替する。ランが終了す

ると、指揮者 UI 側では `drawRunResultPanel()` が毎フレーム `bearWin.hasResult()` を確認し、結果がある場合に獲得コインとランキングを画面上へ表示する。これにより、演奏中の設定変更を防ぎつつ、演奏終了後は再び設定入力可能な状態へ戻るとともに、演奏のテンポに連動した視覚的な演出とその結果のフィードバックを実現している。

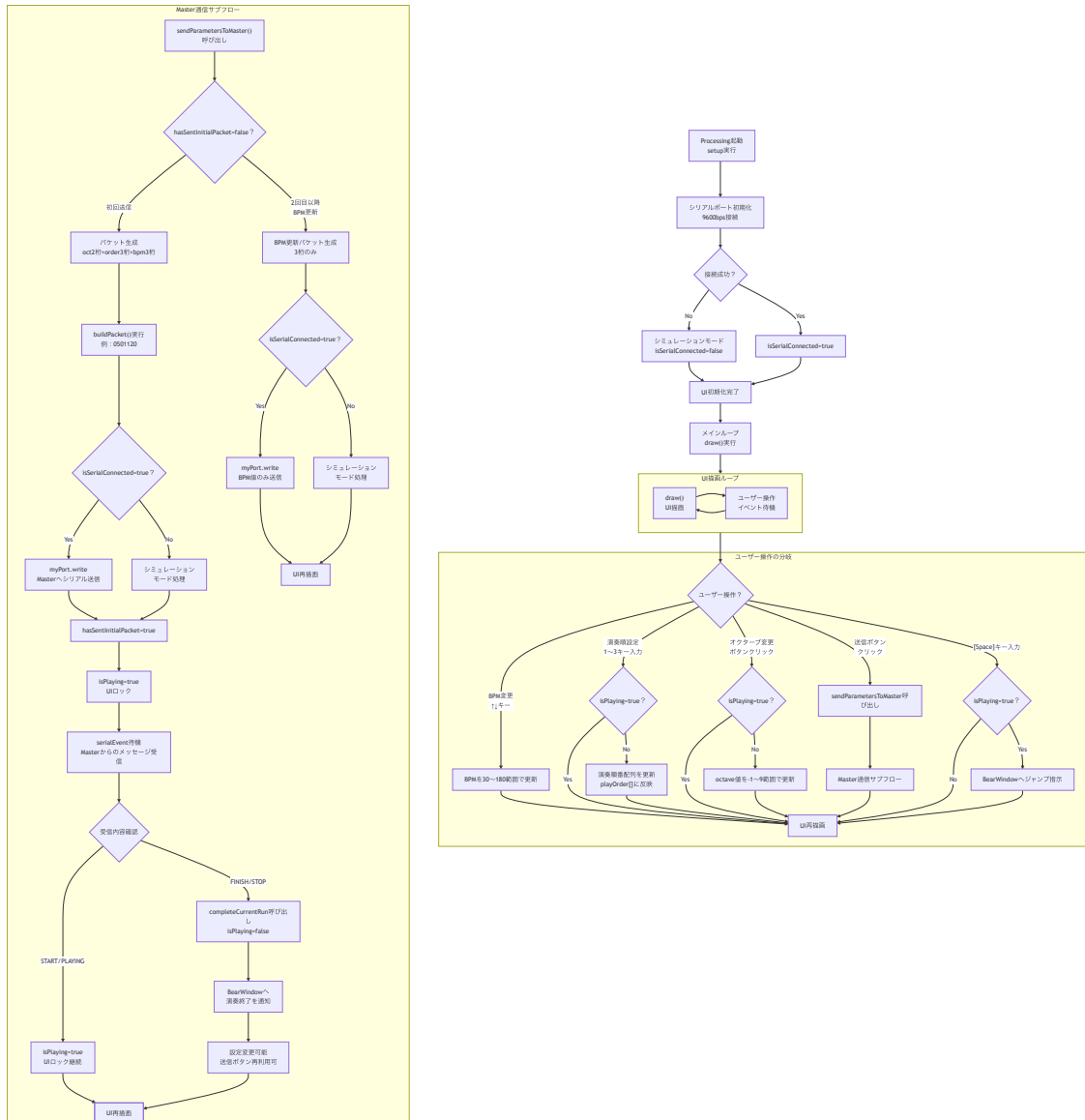


図 6 UI, Master への送信制御フローチャート

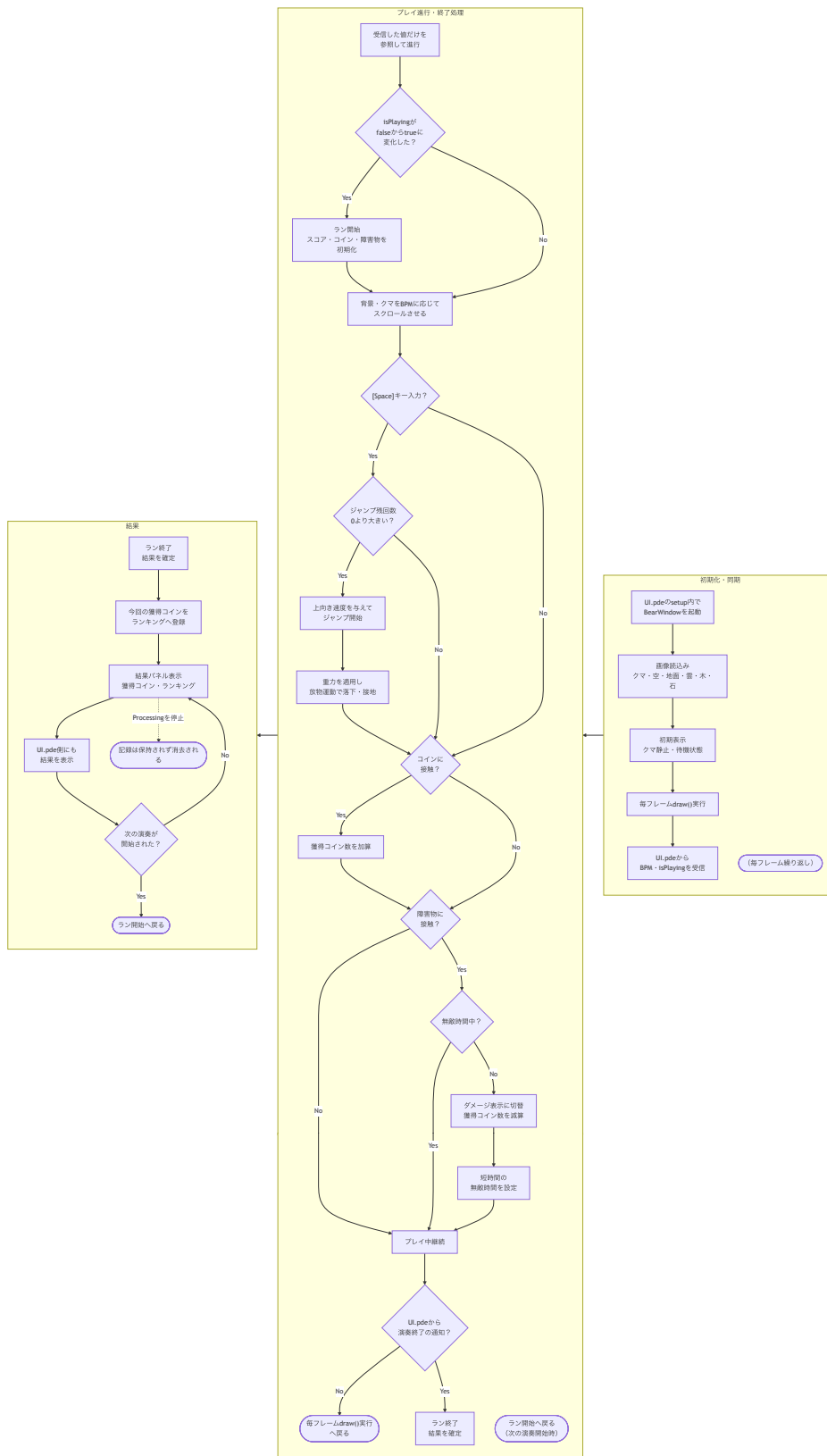


図7 クマアニメーション同期のフローチャート

2.1.2 主要関数表

表 5 に示した主要関数について、制御の流れに沿ってその役割を述べる。UI.pde の `setup()` では、シリアル接続の確立に続いて画像の読み込みと BearWindow の起動を行う。draw() は毎フレーム呼び出される唯一の描画起点であり、UI 各エリアの描画に加えて、`buildPacket()` による送信パケットのプレビュー表示、BearWindow への BPM・演奏状態の同期、および `drawRunResultPanel()` による結果パネルの描画をまとめて行う。キー入力 は `keyPressed()` と `keyReleased()` に分離されており、BPM 変更、演奏順設定、ジャンプ発火、シミュレーション時の終了操作 (s キー) をそれぞれ判定する。マウス操作は `mousePressed()` に集約され、送信ボタン、リセットボタン、オクターブ増減ボタンへのクリックをホバー判定関数 `isHover()` を用いて検出する。ランの開始と終了は、UI.pde 側の `completeCurrentRun()` と、BearWindow 側の `startRun()`、`completeRun()` が対になって管理する。`completeCurrentRun()` は `isPlaying` の解除と BearWindow への終了通知を 1 か所に集約するための関数であり、シリアル受信時とシミュレーション時の両方から呼び出される。BearWindow 内部では、`updateWorld()` が毎フレームの状態更新 (スクロール、ジャンプ、コイン・障害物判定、アニメーション状態) を統括し、draw() が背景・地形・コイン・障害物・クマ・HUD の描画を順に呼び出す。当たり判定は `isCoinCollected()`、`isObstacleHit()`、および円と矩形の判定を行う `circleRectOverlap()` によって実装し、画像のサイズに依存しない独立した判定オブジェクトとして扱う。

表 5: 主要関数表

戻り値	関数名	引数	役割
UI.pde			
void	setup	なし	シリアル接続の確立、フォント設定、クマ画像群の読み込み、BearWindow の起動を行う初期化関数。
void	draw	なし	毎フレーム呼び出され、UI 各エリアの描画、BearWindow への BPM・演奏状態の同期、結果パネルの描画をまとめて行う。
void	keyPressed	なし	↑/↓キーによる BPM 変更、スペースキーによるジャンプ発火、1~3 キーによる演奏順設定、s キーによる終了操作を判定する。
void	keyReleased	なし	スペースキーが離されたことを検知し、ジャンプの押し続け判定フラグを解除する。
void	mousePressed	なし	送信・リセット・オクターブ増減ボタンへのクリックを判定し、対応する処理を呼び出す。
void	serialEvent	port	Master からの受信文字列を解析し、START / PLAYING 時はロック、FINISH / STOP 時はランの終了処理を行う。
boolean	isHover	x, y, w, h	マウス座標が指定した矩形領域内にあるかを判定する。

次ページに続く

表 5 (続き)

戻り値	関数名	引数	役割
void	resetOrder	なし	演奏順番および選択状態を未設定の状態に初期化する。
void	completeCurrentRun	message	isPlaying を解除し、BearWindow 側の completeRun() を呼び出してランを終了させる。
void	drawRunResultPanel	なし	BearWindow にランの結果がある場合に、獲得コインとランキングを指揮者 UI 画面上へ描画する。
String	buildPacket	なし	オクターブ・演奏順・BPM を 8 バイトの固定長文字列にエンコードする。
void	sendParametersToMaster	なし	初回は 8 バイトの初期パケット、以降は BPM のみの 3 バイトパケットを Master へ送信する。
BearWindow.pde			
void	setup	なし	フレームレートの固定、各オブジェクトの初期座標および種類（雲・木）の抽選、コイン・障害物・パーティクルの初期化を行う。
void	draw	なし	毎フレーム呼び出され、updateWorld() による状態更新の後、背景・地形・コイン・障害物・クマ・HUD を順に描画する。
void	updateWorld	なし	BPM に応じたスクロール速度の算出、雲・木・地面・ジャンプ・コイン・障害物の状態更新をまとめて行う。
float	getScrollSpeed	なし	現在の BPM と演奏中フラグから、背景・オブジェクトのスクロール速度を算出する。
void	updateBearState	なし	ダメージ・ジャンプ・歩行のいずれかにクマの表示状態 bearState を更新する。
void	drawBear	x, y, angle	現在の状態に対応するドット絵画像を選択し、足元の補正値を加味してクマを描画する。
PImage	currentBearImage	angle	bearState と歩行位相から、表示すべきクマ画像（歩行 2 種／ジャンプ／ダメージ）を返す。
float	bearHitCenterX	なし	クマの当たり判定円の中心の X 座標を返す。
float	bearHitCenterY	なし	クマの当たり判定円の中心の Y 座標を、ジャンプ量および足元補正値を加味して返す。
void	drawGround	なし	地面画像をオーバーラップさせながら水平方向にタイル状に描画し、継ぎ目のない無限スクロールを実現する。
void	triggerJump	なし	残りジャンプ可能回数が 0 より大きい場合に、初期上向き速度を与えてジャンプを発火する。
void	updateJump	なし	重力による加速度を適用し、クマのジャンプによる上下移動量を更新する。

次ページに続く

表 5 (続き)

戻り値	関数名	引数	役割
boolean	isCoinCollected	i	クマの当たり判定円と指定したコインとの距離から、収集判定を行う。
boolean	isObstacleHit	i	クマの当たり判定円と指定した障害物の矩形との衝突を判定する。
boolean	circleRectOverlap	cx, cy, r, rx, ry, rw, rh	円と矩形の最近接点距離を用いて、円と矩形の衝突を判定する汎用関数。
void	applyObstaclePenalty	i	障害物接触時に獲得コインを減算し、無敵時間の設定とエフェクトの発生を行う。
void	startRun	bpm	今回のランのスコア・コイン・障害物・タイマーをすべて初期化し、ランを開始する。
void	completeRun	なし	ランを終了し、獲得コインをランキング配列へ挿入する。
int	insertRanking	score	渡されたスコアをランキング配列の適切な位置へ挿入し、順位を返す。
String[]	getRankingLines	なし	ランキング配列を表示用の文字列配列に整形して返す。
boolean	hasResult	なし	直前のランの結果が確定済みかどうかを返す。
int	getCurrentRunScore	なし	現在のランで獲得しているコイン数を返す。

2.1.3 主要変数および定数

表 6 に示した主要変数および定数について、システム動作に果たす役割を述べる。UI.pde 側では、現在のテンポを保持する bpm、演奏順を保持する playOrder[], 共通オクターブを保持する octave が、送信パケットを構成する中核の変数である。これらは演奏中フラグ isPlaying の状態に応じて変更可否が切り替わり、isPlaying 自体は Master からのシリアル受信または completeCurrentRun() の呼び出しによって更新される。BearWindow との連携は、参照を保持する bearWin と、スペースキーの押し続けを管理する bearJumpKeyHeld によって行われる。BearWindow.pde 側では、BPM に対するスクロール速度の基準となる BASE_BPM, BASE_SPEED と、ジャンプの物理特性を決める GRAVITY, JUMP_VELOCITY, MAX_JUMPS が描画・操作の基準値となる。クマの位置は BEAR_X, BEAR_GROUND_Y を基準座標とし、画像の見た目と当たり判定の中心とのずれを BEAR_FOOT_OFFSET, TREE_FOOT_OFFSET, OBSTACLE_FOOT_OFFSET で補正する。当たり判定の半径は BEAR_HIT_RADIUS として見た目の画像サイズより小さく設定し、判定の公平性を保っている。コイン・障害物・パーティクルはそれぞれ固定長配列 (coinX[] 等, obstacleX[] 等, scatterX[] 等) で位置と状態を管理し、配列長は COIN_COUNT, OBSTACLE_COUNT, SCATTER_COUNT によって定める。スコア関連では、現在のランで獲得したコイン数を currentRunScore, 過去最高記録を bestScore, 直近 5 回の記録を rankingScores[] に保持する。

表 6: 主要変数および定数表

型	変数名	初期値	役割
UI.pde			
int	bpm	120	現在の設定 BPM を保持する。30～180 の範囲で更新される。
int[]	playOrder	{0, 0, 0}	ピアノ・フルート・木琴の演奏順（0 は未設定）を保持する。
int	octave	4	共通オクターブ（国際式 -1～9）を保持する。
boolean	isPlaying	false	演奏中かどうかを示し、true の間は BPM 以外の変更をロックする。
boolean	hasSentInitialPacket	false	初回設定パケットを送信済みかどうかを示す。
boolean	isSerialConnected	false	Master とのシリアル接続に成功したかどうかを示す。
BearWindow	bearWin	なし	クマアニメーション別ウィンドウへの参照を保持する。
boolean	bearJumpKeyHeld	false	スペースキーの押し続けを判定し、ジャンプの多重発火を防ぐ。
BearWindow.pde			
float	bearBpm	120.0	UI.pde から同期される現在の BPM を保持する。
boolean	bearPlaying	false	UI.pde から同期される演奏中フラグを保持する。
final float	BASE_BPM	120.0	スクロール速度の基準となる BPM 値を保持する。
final float	BASE_SPEED	4.0	基準 BPM 時のスクロール速度を保持する。
final float	GRAVITY	0.9	ジャンプ中にクマへ適用する重力加速度を保持する。
final float	JUMP_VELOCITY	-14.5	ジャンプ発火時の初期上向き速度を保持する。
final int	MAX_JUMPS	2	連続でジャンプ可能な最大回数を保持する（二段ジャンプ）。
final float	BEAR_X	210.0	クマの水平表示位置（画面上で固定）を保持する。
final float	BEAR_GROUND_Y	250.0	クマの接地基準 Y 座標を保持する。
final float	BEAR_FOOT_OFFSET	65.0	クマ画像の足元位置と接地基準 Y 座標とのずれを補正する。
final float	TREE_FOOT_OFFSET	35.0	木の画像の根元位置を地面ラインに合わせるための補正值。
final float	OBSTACLE_FOOT_OFFSET	40.0	障害物（石）の表示位置を補正する値（当たり判定には影響しない）。
final float	BEAR_HIT_RADIUS	30.0	クマの当たり判定円の半径を保持する。
int	bearState	BEAR_STATE_WALK	クマの現在の表示状態（歩行／ジャンプ／ダメージ）を保持する。

次ページに続く

表 6 (続き)

型	変数名	初期値	役割
float	legAngle	0	歩行アニメーションの位相を保持し、BPM に応じて進行速度が変わる。
float	bearJumpY	0	クマのジャンプによる上下方向の変位量を保持する。
int	jumpsRemaining	MAX_JUMPS	接地までに残っているジャンプ可能回数を保持する。
final int	COIN_COUNT	7	画面内に存在させるコインの個数を保持する。
final int	OBSTACLE_COUNT	4	画面内に存在させる障害物の個数を保持する。
int	currentRunScore	0	現在のランで獲得したコイン数を保持する。
int	bestScore	0	起動後の最高獲得コイン数を保持する。
int[]	rankingScores	{0,0,0,0,0}	直近のラン結果上位 5 件のスコアを保持する。
boolean	runFinished	false	現在のランが終了し、結果が確定済みかどうかを示す。

3 付録

- ・添付資料：
 - 回路図：

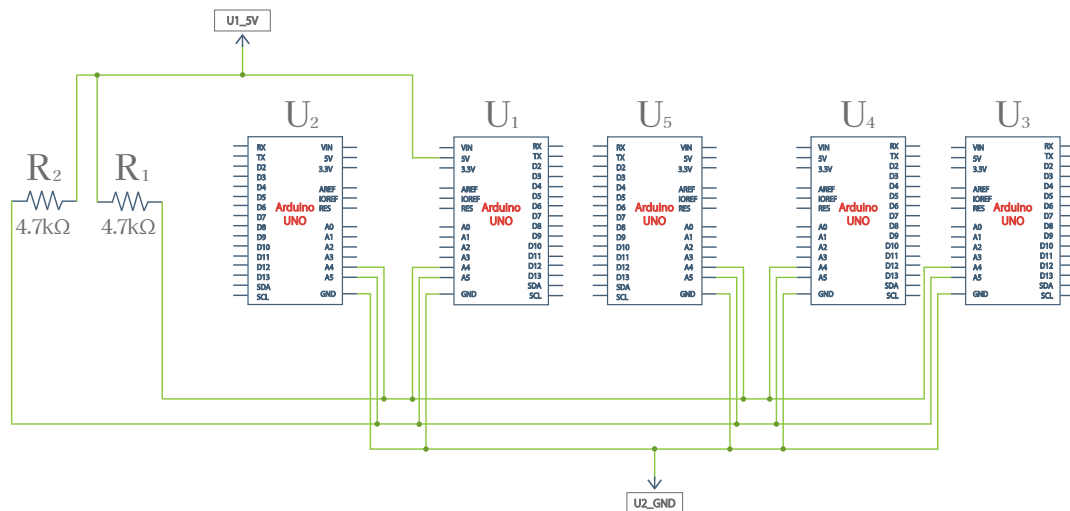


図 8 Arduino オーケストラの回路図

- ソースコード：
 - メインリポジトリ：https://github.com/GingerAle0220/HCK01_group7

UI およびクマさんアニメーション：https://github.com/GingerAle0220/HCK01_group7/tree/main/UI_ver.3

設計書_UI：https://github.com/GingerAle0220/HCK01_group7/tree/main/%E8%A8%AD%E8%A8%88%E6%9B%B8

班員のコード：https://github.com/GingerAle0220/HCK01_group7/tree/main/Ptype1